

Table des matières

Introduction

Système d'exploitation

Ligne de commande

Processus et fils d'exécution

Utilisateurs et groupes

Fichiers

Gestion de la mémoire

Concurrence et parallélisme

Compilation

Environnement logiciel et virtualisation

Environnement logiciel

Notions

- configuration mettant à disposition un ou plusieurs logiciels avec leurs dépendances
 - ex. Ubuntu Linux 24.04 avec Mozilla Firefox 146.0.1 et gcc 13.3

Dépendence

- logiciel dont dépend un autre logiciel pour fonctionner
- ex. la bibliothèque partagée `fiboclever` est une dépendence du programme `fibotest`

Environnement logiciel

Portée

- **environnement global** – logiciels disponibles à tous les utilisateurs d'un système informatique
 - contenu de `/usr/bin`, `/usr/lib`, `/usr/local/bin`, `/usr/local/lib`, `/opt` sur Linux
- **environnement local** – logiciels hérités de l'environnement global combinés aux logiciels disponibles uniquement à un utilisateur ou un groupe d'utilisateurs d'un système informatique
 - ex. logiciels installés dans le dossier personnel d'un utilisateur
- **environnement isolé** – configuration logicielle séparée de l'environnement local répondant à un besoin spécifique tout en évitant des conflits potentiels avec l'environnement local
 - ex. environnement fournissant la version 9 du compilateur `gcc` et séparé de l'environnement local contenant déjà la version 13 de `gcc`
 - ex. environnement fournissant la bibliothèque `fibosimple` et séparé de l'environnement local contenant déjà la bibliothèque `fiboclever`

Environnement logiciel

Durée de vie

- **environnement persistant** – configuration logicielle disponible sans limitation de durée
 - ex. environnement global
- **environnement temporaire** – configuration logicielle mise en place pour effectuer une tâche spécifique et détruite une fois la tâche accomplie
 - ex. test du programme fibotest avec la bibliothèque fiboclever lors d'une séance de travaux pratiques

Environnement logiciel

Déploiement

- manuel
- gestionnaire de paquets
 - gestionnaires de base (Aptitude—commande apt, Homebrew—commande brew, modules)
 - gestionnaires avancés (Guix, Nix, Spack)
 - gestionnaires dédiés (Python – venv et Conda, Caml – Opam)
- conteneurs (Docker, Singularity)

Environnement logiciel

Problème de reproductibilité

Contexte du calcul haute-performance

- environnements logiciels complexes avec de nombreuses dépendences
- déploiement sur plusieurs machines (ordinateurs personnels, centres de calculs, ...)
- déploiement répété à travers le temps (parfois à plusieurs mois ou années d'intervalle)

Problème

- reconstruire exactement le même environnement logiciel (mêmes logiciels, mêmes versions) à travers différentes plateformes et à travers le temps

Motivation

- éviter les incohérences dans les résultats du calcul dues aux changements dans l'environnement logiciel (nouvelle version d'une bibliothèque, du noyau du système d'exploitation, ...)

Déploiement d'un environnement logiciel

Déploiement manuel

- compilation du code source ou acquisition des binaires pré-compilés d'une application et de ses dépendances
- installation à l'emplacement souhaité et configuration

Avantages

- contrôle fin sur l'ensemble du processus (provenance des logiciels, choix des versions, ...)

Inconvénients

- très chronophage (compilation, configuration, tests, ...)
- sujet aux erreurs

Déploiement d'un environnement logiciel

Utilisation d'un gestionnaire de paquets

- délégation de l'acquisition des binaires pré-compilés ou du code source, puis de l'installation et de la configuration à un logiciel spécialisé
- installation automatique à l'emplacement prévu par le gestionnaire de paquets
 - ex. `/usr/bin` pour Aptitude, `/gnu/store` pour Guix

Avantages

- simplification et accélération de l'ensemble du processus
- gestion d'erreurs (dépendance non satisfaite, problème de téléchargement, ...)
- gestionnaires avancés
 - garanties sur la reproductibilité, isolation complète ou partielle de l'environnement local, ...

Déploiement d'un environnement logiciel

Utilisation d'un gestionnaire de paquets

Inconvénients

- gestionnaires de base
 - perte de contrôle à cause de l'automatisation (provenance des logiciels, choix des versions, ...)
 - offre et maintenance des paquets disponibles dépend des développeurs du gestionnaire
 - reproductibilité très limitée (disponibilité des paquets et des versions change d'une distribution Linux à l'autre, d'une version du système d'exploitation à l'autre)
 - ex. Aptitude sur Ubuntu 24.04 ne fournit pas la version 9 de gcc
- gestionnaires avancés
 - complexité d'utilisation élevée

Déploiement d'un environnement logiciel

Utilisation d'un conteneur

- rédaction d'une recette de construction (séquence d'instructions permettant de télécharger, compiler, tester, installer et configurer un ensemble de logiciels)
- construction et empaquetage d'un environnement entier dans une image unique et relocalisable d'une machine à l'autre

Avantages

- possibilité de reproduire le même environnement à travers différentes machines et à travers le temps à condition de préserver l'image contenant l'environnement
- isolation complète ou partielle de l'environnement logiciel déployé

Inconvénients

- déplacement d'images potentiellement très grandes (souvent plusieurs gigaoctets) en fonction des logiciels déployés
- perte de la reproductibilité si les images sont modifiées après leurs construction initiale

Déploiement d'un environnement logiciel

En calcul haute-performance

Besoins

- contrôle fin sur les composants de l'environnement logiciel (provenance des logiciels, versions, options de compilation, configurations)
- reproductibilité de l'environnement logiciel à travers différentes machines et à travers le temps

Approches

- utilisation d'un gestionnaire de paquets avancé
- utilisation des conteneurs

Déploiement d'un environnement logiciel

Exemple illustratif et mise en pratique

« Tutoriel Guix et Org mode – ComPAS 2025 » :

- atelier de mise en pratique autour des problématiques du déploiement logiciel en calcul haute-performance
- animé à l'occasion de la Conférence francophone d'informatique en Parallélisme, Architecture et Système à Bordeaux en été 2025
- réalisable en toute autonomie
- <https://guix-org-tutorial-compas-2025.gitlab.io/tutorial/>

Déploiement d'un environnement logiciel

Exemple illustratif et mise en pratique

« Tutoriel Guix et Org mode – ComPAS 2025 » :

Mise en pratique

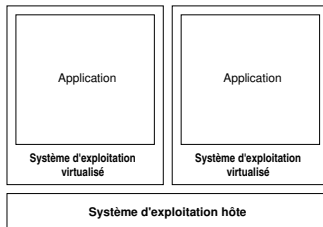
- en suivant les instructions dans le fichier README.md, reproduisez l'environnement logiciel de l'étude <https://gitlab.com/guix-org-tutorial-compas-2025/studies/reference-study> nécessaire à la réalisation des expériences numériques associées grâce au conteneur Docker fourni
 - illustration et mise en pratique de l'utilisation de conteneurs et comparaison avec une procédure de déploiement manuel

Aller plus loin

- lisez le tutoriel <https://guix-org-tutorial-compas-2025.gitlab.io/tutorial/> (sections 1 à 5) et étudiez le déploiement logiciel à l'aide d'un gestionnaire de paquets avancé en vous appuyant sur l'exemple de cette étude (commencez par le fichier README.md)
<https://gitlab.com/guix-org-tutorial-compas-2025/studies/guix-org-study>

Virtualisation

Notions



Machine virtuelle

- émulation d'une machine réelle (aspects matériel et logiciel) de façon logicielle

Motivation

- exécution isolée d'un programme pour des raisons de sécurité, de dépendence sur un matériel spécifique

Virtualisation

Ouverture

Avantages

- regroupement d'applications peu exigeantes en ressources sur une même machine physique
- isolation forte (fichiers, matériel, ...)
- réplication et sauvegarde facilitées

Inconvénients

- abstraction coûteuse comparé au déploiement d'un environnement logiciel isolé
- en cas de faille d'une machine physique, toutes les machines virtuelles qu'elle abrite tombent également en panne